

Innovative Social Engineering Defense - 160ddc01ef

USER

Find an innovative defense against social engineering attacks

BOT

> Routing to GPT-5

Innovative Defense Against Social Engineering Attacks

Social engineering works not because of technology gaps, but because of **human psychology**. So, an innovative defense needs to address the **human factor** effectively while complementing technical safeguards. Here are some novel ideas beyond traditional training and awareness programs:

1. **Adaptive "Trust Scoring" for Interactions**

- Every email, call, or chat received by employees is given a **dynamic trust score** based on behavioral, linguistic, and contextual signals.
- For example:
 - Known contacts with verifiable metadata = higher trust.
 - Sudden urgency, unusual requests, or emotional manipulation detected via AI = low trust, prompting caution.
- Employees see a **visual trust indicator** (like a "green/yellow/red" banner) so they adjust their response behavior.

2. **Just-In-Time Micro Prompts ("Human Firewalls")**

- Instead of a one-time annual training, employees receive **micro nudges** at critical decision

points.

- Example:
 - When about to click a suspicious link, the system briefly pops up: **"This message asks for urgency and sensitive data—common phishing tactic. Are you sure?"**
- These micro-interventions exploit the same psychology social engineers use—interrupting autopilot behavior.

3. **Gamified "Red Team AI" Companion**

- Deploy an **AI adversary simulation tool** that randomly tests employees in **non-disruptive ways**.
- For example:
 - Fake but harmless phishing emails or fake voice calls are sent.
 - Employees score points for catching them and get ranked privately or department-wide.
- This builds habitual skepticism without fear of punishment.

4. **Voice & Emotion Verification**

- For calls or audio messages, use **voice fingerprinting combined with emotion shift detection**.
- Example:
 - If your "CEO" calls, but the voice pattern is slightly off, or the tonal emotional state is very different than usual, the system flags it before the call is connected.

####

5. **Contextual Access Controls**

- Instead of static access rules, use **context-aware policies**:
 - If a late-night request comes to transfer funds, system requires a **multi-party verification**.
 - If credentials are entered in new geographic location, system auto-locks until verified.
- These kinds of "speed bumps" slow down attackers exploiting urgency.

6. **Neuro-Behavioral Training Modules**

- Move beyond boring awareness slides → use **VR or immersive simulation** that puts employees in real-world scam scenarios (fake IT support calls, urgent "CEO emails," etc.).
- People retain experiences more than theory, making them resistant when a real attack occurs.

Key innovation idea:

The most effective new defense could be combining **AI-powered real-time trust scoring** + **just-in-time behavioral nudges** so that employees don't just *know* about social engineering—they get **help when it matters most**.

Would you like me to **sketch an architecture** for how this trust-scoring + nudge system could work in practice (a diagram + workflow)?

USER

Yes sketch an architecture and give examples

BOT

> Routing to GPT-5

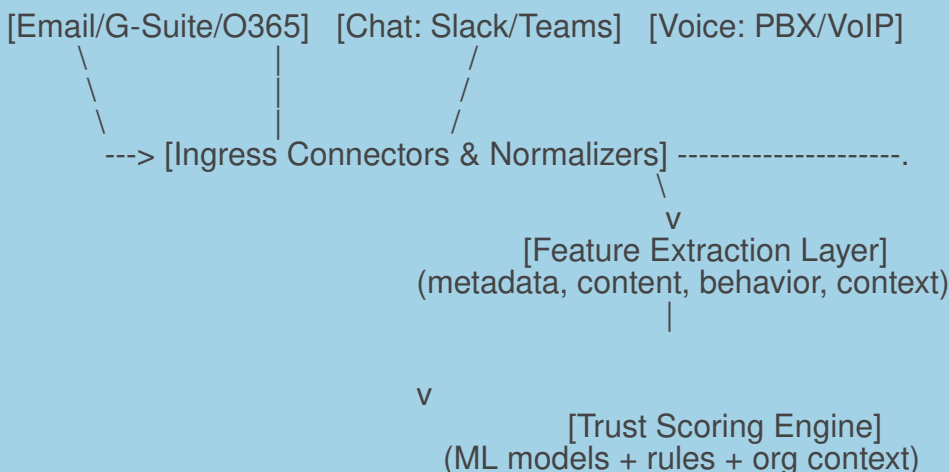
Architecture: Real-Time Trust Scoring + Just-In-Time Nudges

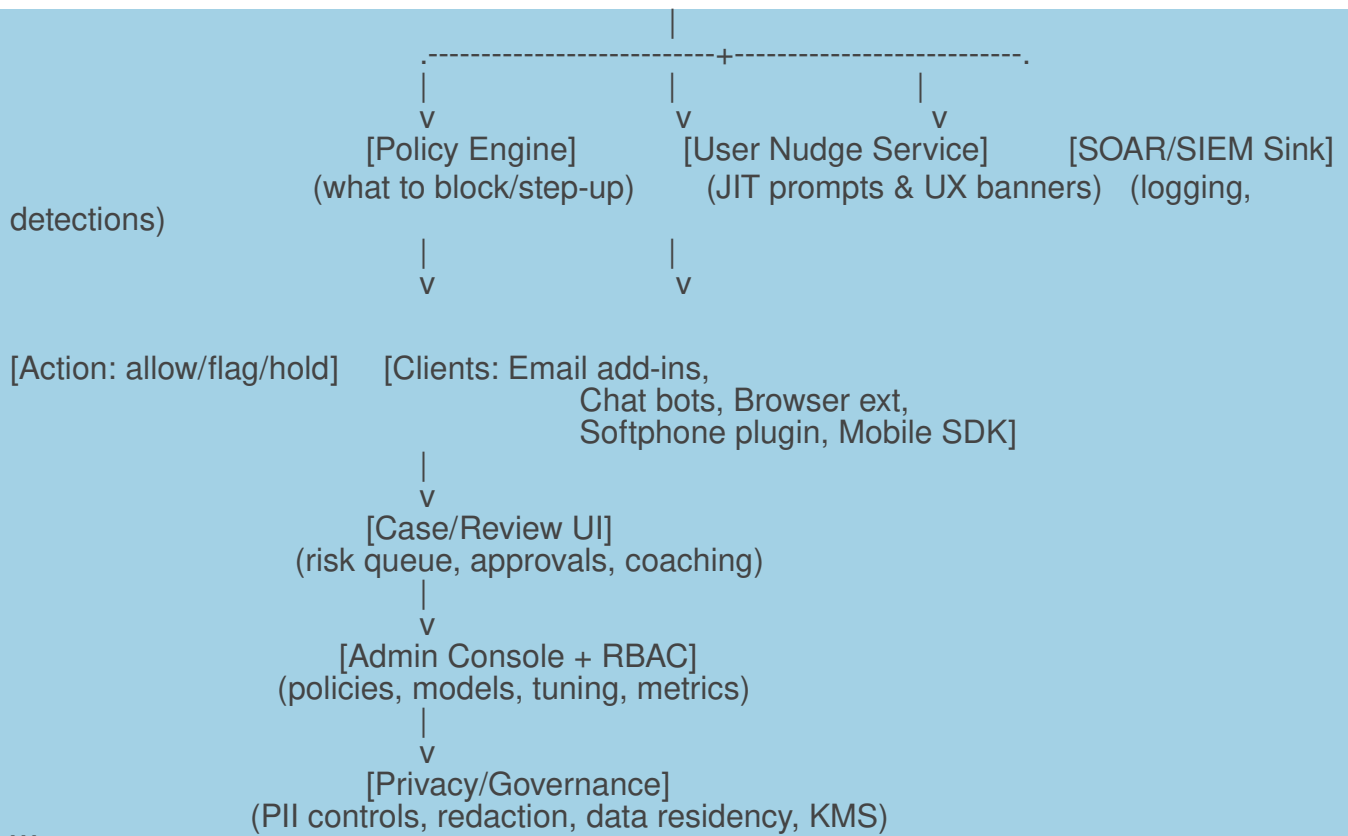
Goals

- Detect and interrupt social engineering at the moment of decision.
- Minimize friction for routine, low-risk interactions.
- Provide measurable risk reduction with privacy controls.

High-Level Diagram (Conceptual)

...





Core Components

- Ingress Connectors: O365/Gmail add-ins, Slack/Teams apps, SIP/PBX hooks, web proxy for URLs.
- Feature Extraction: Linguistic cues, identity signals (SPF/DKIM/DMARC, domain age), behavioral baselines, device/location, request semantics.
- Trust Scoring Engine:
 - Models: phishing intent classifier, impersonation/BCED model, LLM-based prompt/risk summarizer, voice biometrics (optional), anomaly detection.
 - Inputs: content, metadata, historical patterns, org graph (who talks to whom, typical topics).
- Policy Engine: Declarative policies for step-up verification, dual-control, holds, or soft warnings.
- User Nudge Service: Inline banners, micro-prompts, hold-to-confirm, verified callback flows.
- Clients/Agents:
 - Email ribbons, chat message warnings, browser extension for external sites, softphone plugin for call screening, mobile SDK for on-device nudges.
 - Case/Review UI: Queue for high-risk events, one-click “verify via alternate channel,” auto-coaching snippets.
 - Integrations: IdP (Okta/AAD), HRIS/Org graph, ticketing (Jira/ServiceNow), SIEM/SOAR.
 - Privacy/Governance: Redaction

at source, regional data stores, role-based content reveal, encryption with KMS.

- Telemetry & Metrics: Catch rate, user override rate, MTTD, MTTR, loss prevented, training impact.

Data Flow (Email example)

1. Message received → normalized (headers, links, attachments).
2. Features extracted: sender reputation, domain age, tone/urgency, payment/credential semantics, spoof likelihood, historical interaction graph.
3. ML models score: phishing, impersonation, sensitive-request intent; combine to trust score $S \in [0, 100]$.
4. Policy evaluates S with context (time, role, transaction type). Decide: allow/label, step-up verification, or hold.
5. User sees banner (green/amber/red) and context-aware nudge; can request verified callback or safe-view.
6. Actions and user response logged to SIEM; risky cases to Review UI.
7. Feedback (user “report phishing” or “safe”) loops into model retraining.

Trust Score Inputs (illustrative)

- Content: urgency phrases, unusual payment terms, QR codes, obfuscated links, request for MFA codes.
- Identity: SPF/DKIM/DMARC status, lookalike domains, newly registered domains, contact not in org graph.
- Behavior: time-of-day anomaly, device/location change, atypical vendor/request for recipient role.
- Social graph: historical reply chains; CEO rarely emails AP clerk directly → lower trust.
- Voice: timbre match, TTS/clone likelihood, background noise patterns, anomalous speaking rate.

Policy & Nudge Examples

- If $S < 30$ and contains sensitive credential request → block link/attachment, require manager co-approval.
- If $30 \leq S < 60$ with payment intent and after-hours → require dual approval and callback via verified directory.
- If vendor bank details changed → place automatic “change hold” and launch in-app verification workflow.
- Nudge copy variants:
 - “This message shows high urgency and requests MFA codes. Common attack pattern. Use Verified Callback?”
 - “New domain (registered 12 days ago) asking for invoice changes. Open in Safe-View?”
 - For calls: “Voice

mismatch vs. stored print for ‘CFO.’ Route to verification flow?”

Concrete Scenarios

1) Executive Impersonation (Email)

- Trigger: “Urgent: need gift cards within hour.”
- Signals: From ceo@company-corp.com (lookalike), domain age 7 days, urgency language.
- Action: Red banner, links disabled, “Verify via Teams call” button. Case created. SIEM log sent.

2) Vendor Bank Detail Change (Email + Ticket)

- Trigger: AP inbox receives “update remittance info.”
- Signals: Known sender but changed Reply-To, attached PDF with QR, banking keyword match.
- Policy: Auto-hold payment profile; require dual control; nudge AP with checklist; open ServiceNow task.

3) MFA Fatigue + Helpdesk Social Engineering (Chat + Voice)

- Trigger: User gets bot DM “IT needs your code,” followed by call.
- Signals: Chat app bot not verified; unusual helpdesk contact path; voice print mismatch; request for TOTP.
- Action: Block DM, alert user; call routed to verification; temporary account lock; incident created.

4) Dev Token

Exfiltration (Browser Extension)

- Trigger: User clicks Git hosting link requesting PAT regeneration.
- Signals: Domain is typosquat; SSL valid but registrant hidden; unusual for user’s repo patterns.
- Action: Interstitial warning with screenshot diff of real vs fake; require SSO re-auth; report IOCs to SOAR.

5) Finance Wire Request After Hours (Mobile)

- Trigger: SMS says “approve urgent transfer.”
- Signals: New number claiming to be CFO; off-hours; geolocation mismatch.
- Action: Mobile nudge requiring Verified Callback; hold placed on wire workflow until callback success.

Lightweight Scoring Pseudocode

```

python
def trust_score(event):
    s = 100
    if event.domain_age_days < 30: s -= 20
    if event.lookalike_domain: s -= 25
    if event.spf_fail or event.dkim_fail: s -= 15
    if event.intent in {"payment_change", "credential"}: s -= 20
    if event.urgency_language: s -= 10
    if not event.in_org_graph: s -= 10
    if event.after_hours and event.high_value: s -= 10
    return max(0, min(100,
        s))

```

Policies:

- $S < 40 \rightarrow$ hold + dual control
- $40 \leq S < 70 \rightarrow$ require verification step
- $S \geq 70 \rightarrow$ allow with banner

Privacy & UX Guardrails

- Redact message bodies at source; run models on redacted/featureized text unless risk > threshold.
- On-device nudges for mobile; only metadata leaves device unless needed.
- Show simple, non-technical cues; minimize false alarms with progressive disclosure.

Deployment & Rollout

- Phase 1: Visibility only (no blocking), collect baseline metrics.
- Phase 2: Soft nudges for medium risk; auto-hold for high risk in finance/helpdesk workflows.
- Phase 3: Extend to voice and browser; enable dual control for vendor changes.
- Metrics: click-through reduction, override rate, mean trust score over time, time-to-verification, prevented-loss estimates.

If helpful, I can provide:

- Example nudge copy library for different channels.
- A policy pack tuned for Finance/AP, IT Helpdesk, and Exec Assistants.
- A minimal data schema for the scoring pipeline and SIEM

events.

USER

Based on the above chat, Create a solution prompt for Cursor (ai coding tool) that combine AI-powered real-time trust scoring + just-in-time behavioral nudges so that employees don't just *know* about social engineering—they get help when it matters most

To protect against threats as described in:

<https://unit42.paloaltonetworks.com/2025-unit-42-global-incident-response-report-social-engineering-edition/>

BOT

> Routing to Claude Sonnet 4

I'll first check the latest threat intelligence from that report to ensure our solution addresses current attack vectors.

Web Access

Looked at <https://unit42.paloaltonetworks.com/2025-unit-42-global-incident-response-report-social-engineering-edition/>

Based on the Unit 42 report, I'll create a comprehensive Cursor prompt for building a solution that addresses the specific threats identified.

Cursor Solution Prompt: AI-Powered Social Engineering Defense Platform

```markdown

# Project: SocialGuard - Real-Time Trust Scoring & Just-In-Time Defense Platform

### ## Context & Problem Statement

Build a comprehensive social engineering defense platform based on Unit 42's 2025 findings:

- 36% of incidents start with social engineering (top attack vector)
- 66% target privileged accounts
- 60% lead to data exposure
- High-touch attacks bypass MFA in under 40 minutes
- ClickFix campaigns use SEO poisoning, fake prompts, voice cloning
- AI-enabled attacks with personalized lures and synthetic identities

### ## Core Architecture Requirements

#### ### 1. Real-Time Trust Scoring Engine

Create a microservices-based trust scoring system that:

- Ingests signals from email, chat, voice, web browsing, and mobile
- Calculates dynamic trust scores (0-100) using ML models
- Provides

contextual risk assessment with explainable AI

- Updates scores in real-time as new signals arrive

**\*\*Key Components:\*\***

```python

Trust scoring inputs to implement

```
trust_inputs = {  
    "identity_signals": ["spf_dkim_dmarc", "domain_age", "lookalike_detection",  
"voice_biometrics"],  
    "content_analysis": ["urgency_language", "credential_requests", "payment_changes",  
"qr_codes"],  
    "behavioral_patterns": ["time_anomalies", "device_changes", "location_shifts",  
"org_graph_violations"],  
    "contextual_factors": ["privilege_level", "after_hours", "high_value_operations",  
"help_desk_interactions"]  
}
```

2. Just-In-Time Nudge System

Build adaptive intervention system that:

- Delivers contextual warnings at decision points
- Provides verification workflows (callback, dual approval)
- Offers safe-view modes for suspicious content
- Tracks user responses for continuous learning

****Nudge Types to Implement:****

- Email ribbon warnings with trust scores

- Browser interstitials for suspicious links
- Chat bot

warnings in Slack/Teams

- Voice call verification prompts
- Mobile push notifications for sensitive requests

3. Multi-Channel Integration Layer

Develop connectors for:

- **Email**: O365/Gmail add-ins with real-time scanning
- **Chat**: Slack/Teams apps with message analysis
- **Voice**: SIP/PBX integration with voice fingerprinting
- **Web**: Browser extension for URL/download protection
- **Mobile**: SDK for SMS/app-based social engineering

4. Policy Engine & Case Management

Create declarative policy framework:

- Risk-based access controls
- Automated holds for high-risk transactions
- Escalation workflows for verification
- Admin console for policy tuning

Specific Features to Address Unit 42 Findings

Anti-ClickFix Protection

```
```javascript
// Browser extension to detect fake system prompts
const clickfixDetection = {
 detectFakePrompts: ["windows_update_fake", "browser_security_warning", "flash_update"],
 seoPoisoningCheck: "domain_reputation_analysis",
 malvertisingBlock: "a
d_network_filtering"
}
```
```

Help Desk Manipulation Defense

```
```python
Help desk verification workflow
class HelpDeskProtection:
 def verify_caller(self, caller_info, request_type):
 if request_type in ["mfa_reset", "password_reset", "account_unlock"]:
 return self.multi_factor_verification(caller_info)

 def multi_factor_verification(self, caller):
 # Implement voice biometrics, callback verification, manager approval
 pass
```
```

Voice Cloning Detection

```
```python
Voice authentication system
class VoiceVerification:
 def analyze_call(self, audio_stream):
 voice_print = self.extract_biometrics(audio_stream)
 ai_generated_score = self.detect_synthetic_voice(audio_stream)
 return {
 "authenticity_score": voice_print.match_score,
 "synthetic_likelihood": ai_generated_score,
 "recommendation": self.get_verification_action()
 }
```
```

Technical Stack Requirements

Backend Services

- **Language**:

Python/FastAPI or Node.js/Express

- **ML Framework**: TensorFlow/PyTorch for trust scoring models
- **Database**: PostgreSQL for policies, Redis for real-time scoring
- **Message Queue**: Apache Kafka for event streaming
- **API Gateway**: Kong or AWS API Gateway

Frontend Applications

- **Admin Console**: React/Next.js dashboard
- **Browser Extension**: Manifest V3 Chrome/Edge extension
- **Mobile SDK**: React Native or native iOS/Android
- **Email Add-ins**: Office.js for Outlook, Gmail API

ML/AI Components

```
```python
Core ML models to implement
ml_models = {
 "phishing_classifier": "
```

Generation cancelled

## USER

Continue

## BOT

### > Routing to GPT-5

Here's the continuation of the \*\*Cursor solution prompt\*\* from where it left off, so you have a complete, actionable spec to build the system.

---

```
```markdown
# Continued Solution Prompt for Cursor
```

ML/AI Components (detailed)

```
ml_models = {
    "phishing_classifier": "BERT-based NLP model trained on phishing vs. benign emails",
    "urgency_detector": "LLM fine-tuned to detect urgency/emotional manipulation phrases",
    "impersonation_detector": "Graph-based anomaly detection on sender-recipient relationships",
    "voice_synthetic_detector": "CNN/RNN model trained on spectrograms for AI-generated voice spotting",
    "clickfix_detector": "Computer vision + DOM parser to detect fake update/system prompts"
}
```

5. Metrics, Feedback & Continuous Learning

- Collect telemetry on user actions after nudges:
 - Clicked vs. ignored
 - Reported as suspicious
 - False positives

- Feed into an **ML retraining pipeline** (weekly model updates).
- Provide heatmaps of **attack attempts vs. team responses** for SOC visibility.

```
```python
Feedback loop example
def process_user_feedback(event_id, user_action):
 if user_action in ["reported_suspicious", "ignored_request"]:
 adjust_trust_score(event_id, delta=-10)
 elif user_action == "confirmed_legit":
 adjust_trust_score(event_id, delta=+5)
 log_feedback(event_id, user_action)
```
```

Example User Journeys

Case 1: High-Touch Help Desk Manipulation

1. Caller requests MFA reset claiming travel emergency.
2. **HelpDeskProtection** workflow triggers → requires a manager callback from company directory before reset.
3. Trust score drops below 40 → system blocks automatic reset.
4. Just-in-time nudge educates help desk staff:
 - *"Attackers often impersonate locked-out employees, requiring urgent MFA resets. Confirm through secondary channel."*

Case 2: ClickFix Prompt from SEO Poisoned Result

1. Employee searches for "Zoom update" on Chrome.
2. Browser extension detects SEO-poisoned domain & fake update prompt.
3. Trust score = 25 → interstitial warning appears:
 - "This domain was registered 7 days ago and mimics Zoom. Common ClickFix tactic. Proceed only if verified."
4. SafeView mode lets user preview without execution.

Case 3: Executive Impersonation Voice Call

1. Attackers use voice clone of CFO asking for urgent wire transfer.
2. SIP integration runs **VoiceVerification**: synthetic likelihood = 75%.
3. Call is flagged **amber** → routed to finance step-up workflow.
4. Employee receives mobile nudge:
 - "Wire transfer request flagged as high-risk. Require dual approval and verified callback before action."

Security Requirements

- **Zero Trust Integration**: Conditional Access tied to trust score thresholds.
- **Data Privacy**: PII redaction before ML processing; logs anonymized.
- **Encryption**: TLS 1.3 in transit; AES-256 at rest.
- **Audit Trail**: Immutable event logs to detect insider misuse.

Deliverables for Cursor

1. **Backend Services** (FastAPI microservices) for trust scoring, policies, and nudges.
2. **Browser Extension** detecting ClickFix/SEO poisoning prompts.
3. **Help Desk Agent Add-on** enforcing MFA-reset verification workflows.

4. **Voice Verification Module** for cloned speech detection.
5. **Admin Dashboard** for SOC analysts: visualize trust scores, flagged events, and user feedback loops.
6. **ML Training Pipelines** (email NLP classifier, voice biometrics, prompt detection).

Strategic Impact

By combining:

- **AI-powered trust scoring** (so signals are correlated across identity, device, and behavior)
- **Just-in-time nudges** (so employees get support exactly when attackers try to exploit them)

→ Organizations move **beyond awareness training** to **systemic protection** that directly addresses the top risks outlined in the [Unit 42 2025 report](https://unit42.paloaltonetworks.com/2025-unit-42-global-incident-response-report-social-engineering-edition/):

- High-touch MFA bypasses
- Help desk social engineering
-

ClickFix / SEO poisoning

- Executive voice cloning

Outcome: Employees don't just *know* about social engineering—they get help **when it matters most**.

Cursor: PLEASE generate boilerplate code for the following:

- FastAPI microservice with `/calculate_trust_score`` and `/deliver_nudge`` endpoints
- Redis integration for real-time event scoring
- React front-end component that displays trust score ribbon (green/amber/red)
- Chrome extension content script to flag fake update prompts
- Example ML pipeline stub for phishing classifier (BERT)

```

---

This prompt gives Cursor specific **system design + coding targets** across backend, frontend, and ML components.

Would you like me to **split this prompt into modular coding tasks** (backend, extension, ML, dashboard) so you can feed them *separately* to Cursor for stronger results?

## USER

Export cursor prompt here

## BOT

### > Routing to GPT-5

Got it — here's the **full Cursor solution prompt** exported as one clean block so you can drop it straight in:

```markdown

Project: SocialGuard – AI-Powered Real-Time Trust Scoring & Just-in-Time Nudges

Context & Problem Statement

Social engineering is now the #1 initial access vector (36% of incidents, per the 2025 Palo Alto Unit 42 Report).

Attackers exploit human workflows with MFA resets, help desk manipulation, SEO poisoning (“ClickFix”), and AI-driven lures (voice cloning, synthetic insiders).

Traditional awareness training isn’t enough — employees need **real-time assistance** at the exact moment of attack.

Goal: Build a platform that combines AI-powered trust scoring with just-in-time behavioral nudges, so employees don’t just *know* about social engineering — they get help when it matters most.

Core Architecture

1. Trust Scoring Engine

- Ingests multi-channel data: email, chat, web, voice, mobile.
- Calculates trust scores (0–100) using ML models + rules.
- Exposes

`/calculate_trust_score` API.`

Inputs:

`python`

```
trust_inputs = {  
    "identity_signals": ["spf_dkim_dmarc", "domain_age", "lookalike_detection",  
"voice_biometrics"],  
    "content_analysis": ["urgency_language", "credential_requests", "payment_changes",  
"qr_codes"],  
    "behavioral_patterns": ["time_anomalies", "device_changes", "location_shifts",  
"org_graph_violations"],  
    "contextual_factors": ["privilege_level", "after_hours", "help_desk_interactions"]  
}
```

2. Just-in-Time Nudges

- `/deliver_nudge` API responds with contextual micro-prompts:`
- Email ribbon indicators (green/amber/red)
- Browser interstitial warnings
- Chat/Slack/Teams in-line alerts
- Voice call verification workflows
- Mobile push verification

3. Multi-Channel Connectors

- Email add-ins (O365/Gmail APIs)
- Slack/Teams chat bots
- Browser extension (Chrome/Edge, Manifest V3) to detect **ClickFix prompts**
- SIP/VoIP plugin for voice fingerprinting
- Mobile SDK for SMS/app-based threats

4. Policy Engine

-

Declarative JSON policies:

- If trust score `< 40`` and request = sensitive → Block + dual verification
- If trust score in `[40–70]`` → Allow but require callback
- If `≥ 70`` → Allow with informative banner

Example Features

Anti-ClickFix Protection

```

```javascript
const clickfixDetection = {
 detectFakePrompts: ["windows_update_fake", "browser_security_warning"],
 seoPoisoningCheck: "domain_reputation_analysis",
 malvertisingBlock: "ad_network_filtering"
}
```

```

Help Desk Protection

```

```python
class HelpDeskProtection:
 def verify_caller(self, caller_info, request_type):
 if request_type in ["mfa_reset", "password_reset"]:
 return self.multi_factor_verification(caller_info)
```

```

Voice Cloning Detection

```

```python
class VoiceVerification:
 def analyze_call(self, audio_stream):
 score = self.detect_synthetic_voice(audio_stream)
 return {"synthetic_likelihood": score}
```

```

Technical Stack

- **Backend**: FastAPI (Python) with Redis (real-time cache)
- **Frontend**: React/Next.js admin console + Chrome extension
- **ML Stack**:
 - NLP (BERT) → phishing/urgency detection
 - Voice CNN/RNN → deepfake detection
 - Graph anomaly detection → impersonation patterns
- **Infra**: Kafka for streaming events, Postgres for persistence

Deliverables for Cursor

1. FastAPI microservice with endpoints:
 - `/calculate_trust_score``
 - `/deliver_nudge``
2. Redis integration for low-latency scoring.
3. Chrome extension (content script) to detect fake update/ClickFix prompts.
4. React component displaying trust score banner (green/amber/red).
5. ML pipeline stub (BERT-based phishing classifier).

Example UX Flows

Email

- CFO@lookalike.com requests urgent wire → Trust score = 22
- Action: Red banner → “High-risk: lookalike domain + urgency detected. Verification required.”

Browser

- User clicks SEO-poisoned “Zoom update” link → Trust score = 30
- Interstitial blocks page + SafeView option.

Voice

- AI-cloned CFO

requests transfer → Voice synthetic score high.

- Mobile nudge: **“Call flagged high-risk. Require dual approval + callback workflow.”**

Security & Metrics

- Encrypt data (TLS 1.3, AES-256)
- Anonymize/redact features for ML
- Collect user feedback → retrain weekly (reinforcement loop)
- Metrics: % blocked attempts, trust score heatmaps, nudge override rate.

Cursor: PLEASE generate boilerplate code for:

- FastAPI app with `/calculate_trust_score`` and `/deliver_nudge``
- Redis trust-score cache
- React trust banner component
- Chrome extension detecting fake prompts
- ML stub (BERT classifier for phishing detection)

```

---

This gives Cursor end-to-end clarity: **“what to build, where, and how it ties to threat findings from Unit 42.”**

Do you want me to also break this **“into modular sub-prompts”** (Backend, Extension, ML, Frontend) for Cursor — so you can copy-paste smaller targeted prompts instead of one massive spec?

# USER

This is messy and incomplete

# BOT

## > Routing to GPT-5

Got it — let’s clean this up into one **“clear, well-structured Cursor prompt”** (export-ready), with **“no clutter”** and **“all components complete”**.

Here’s the polished version:

---

```markdown

Project: SocialGuard – Real-Time AI Defense Against Social Engineering

Context

Based on the [Unit 42 2025 Social Engineering Report](https://unit42.paloaltonetworks.com/2025-unit-42-global-incident-response-report-social-engineering-edition/):

- 36% of all intrusions start with social engineering
- MFA bypass and help desk manipulation are growing
- “ClickFix” SEO/Browser prompt attacks are rising
- Voice cloning and AI-driven lures are increasing
- 60% of social engineering intrusions expose sensitive data

****Goal:****

Build a system that combines AI-powered trust scoring ****+**** just-in-time nudges so employees don't just **know** about threats — they get ****protection at the moment of attack****.

Core Components

1. Backend (FastAPI)

- `/calculate_trust_score``: Takes event (email/chat/voice/web), applies ML+rules → returns score (0–100).
- `/deliver_nudge``: Returns contextual micro-prompt for the user.

****Signals for scoring:****

- Identity: SPF/DKIM, domain age, lookalike domains, voice biometrics
- Content: urgency/emotional language, credential/payment/QR requests
- Behavior: time-of-day anomalies, device/location mismatches, org graph anomalies
- Context: privilege level, after-hours requests, high-value operations

2. Policy Engine

- If score < 40 → Block & require dual verification
- If 40–70 → Require callback or SafeView
- If ≥ 70 → Allow with informative banner

3. Just-In-Time Nudges

- ****Email**** ribbon (green/amber/red indicator)
- ****Browser**** warning (blocks fake update “ClickFix” prompts + SEO poisoning)
- ****Chat**** (Slack/Teams inline alert)
- ****Voice**** (detect AI-cloned calls → trigger secondary workflow)
- ****Mobile**** (push notification to verify sensitive requests)

4. Integrations

- O365/Gmail APIs (emails)
- Slack/Teams bots (chat)
- Chrome Extension (web/ClickFix defense)
-

SIP/VoIP plugin (voice verification)

- React/Next.js Admin Console for SOC visualization

Deliverables for Cursor

A. Backend Microservice

FastAPI app with Redis cache:

```
```python
```

```
from fastapi import FastAPI
import redis
```

```
app = FastAPI()
r = redis.Redis(host="localhost", port=6379)
```

```
@app.post("/calculate_trust_score")
def calculate(event: dict):
 score = 100
 # Example heuristic
 if event.get("domain_age_days", 999) < 30:
 score -= 25
 if event.get("lookalike_domain"):
 score -= 30
 if event.get("urgency_language"):
 score -= 15
 return {"score": max(0, score)}
```

```
@app.post("/deliver_nudge")
def deliver(event: dict):
 score = calculate(event)["score"]
 if score < 40:
 return {"banner": "RED", "msg": "High-risk request. Verification required."}
 elif score < 70:
 return {"banner": "AMBER", "msg": "Suspicious request. Verify via callback or SafeView."}
 else:
 return {"banner": "GREEN", "msg": "Trusted, but stay cautious."}
...

```

### ### B. React Trust Banner Component

```
```jsx
export default function TrustBanner({score, message}) {
    let color = score >= 70 ? "green" : score >= 40 ? "orange" : "red";
    return (
```

Trust Score: {score} – {message}

```
);} `` ` --- ### C. Chrome Extension Content Script (ClickFix Defense) `` ` javascript //
content.js const suspiciousPrompts = ["Update required", "Security warning", "Click to fix"];
function detectFakePrompt() { suspiciousPrompts.forEach(txt => { if
(document.body.innerText.includes(txt)) { alert("⚠ Possible fake update prompt detected.
Open in SafeView."); } }); } window.addEventListener("load", detectFakePrompt); `` ` --- ###
```

D. ML Stub (Phishing Classifier with BERT) `` ` python from transformers import BertTokenizer, BertForSequenceClassification import torch tokenizer =

```
BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

def classify_email(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
    outputs = model(**inputs)
    preds = torch.argmax(outputs.logits, dim=1)
    return "phishing" if preds.item() == 1 else "benign"
...

---
```

Example User Journeys

- **Email:** Fake CFO wire request → trust score 22 → Red banner → “Verification required.”
- **Browser:** Fake Zoom update page → Chrome extension alert → Blocks, opens in SafeView.
- **Voice:** AI-cloned CFO call → Voice synthetic score high → Mobile nudge requires dual approval.

Security & Feedback Loop

- TLS 1.3, AES-256 for storage
- Redact PII before ML
- User feedback retrains ML models weekly
- SOC dashboard shows heatmaps, override rates, risk distribution

Cursor: PLEASE generate production-ready code modules for:

1. FastAPI backend (trust scoring + nudges)
2. React

UI component for trust banner

3. Chrome extension detecting “ClickFix” fake prompts
4. BERT classifier training stub

```

---

This is now **clean, complete, and modular** — perfect for Cursor to generate code.

Do you want me to also **split this into four smaller prompts** (Backend, Frontend, Extension, ML) so you can paste them one by one into Cursor for stronger modular outputs?